

Algoritmos Genéticos - Problema de las N-Reinas

Lucía Sánchez Caparrós
Lucía Muñoz del Río
Zhibo Daniel Chen
María Victoria Maldonado Bao

0611565598@uma.es
Ciberseguridad e Inteligencia Artificial. Universidad de Málaga.

1 Introducción

El siguiente informe tiene como objetivo documentar cómo se ha llevado a cabo la implementación de un **Algoritmo Genético** que resuelve el famoso problema de las **N-Reinas**. A lo largo de las secciones se explicarán las **definiciones** de individuo, población y función de aptitud (*fitness*), tanto en lenguaje natural como matemático; los **resultados obtenidos** y un análisis de los mismos, así como una comparación entre ellos; y las **conclusiones** que han derivado a partir de las salidas encontradas.

2 Descripción de los Datos

Para abordar la resolución del problema de las N-Reinas mediante un algoritmo genético utilizando la librería DEAP, es necesario definir matemáticamente los conceptos fundamentales del algoritmo: el individuo, la población y la función de aptitud (o fitness). Estas definiciones están directamente ligadas a la representación elegida en la implementación.

2.1 Definición de individuo

Un individuo (o cromosoma) representa una **solución candidata** al problema, es decir, una posible configuración de las reinas en el tablero. En nuestra implementación, para optimizar el espacio de búsqueda, se asume implícitamente que no puede haber dos reinas en la misma columna ni en la misma fila. Por lo tanto, un individuo se representa como un **vector unidimensional** (lista) formado por una permutación de números enteros, donde el índice del array representa la columna y el valor almacenado en dicho índice representa la fila. Matemáticamente, para un tablero de dimensiones $N \times N$, un individuo x se define como un vector:

$$x = (x_0, x_1, \dots, x_{N-1})$$

Donde:

- $x_i \in \{0, 1, \dots, N-1\}$ es la fila en la que se ubica la reina de la columna i .
- Se cumple que $x_i \neq x_j$ para todo $i \neq j$ (representación mediante permutación).

2.2 Definición de población

La población es el **conjunto de individuos** (soluciones candidatas) que existen en una generación concreta del algoritmo.

Si denotamos como M al tamaño de la población (en nuestra ejecución base, $M=100$), la población P en la generación t se define matemáticamente como el conjunto:

$$P(t) = \{x^{(1)}, x^{(2)}, \dots, x^{(M)}\}$$

Donde cada $x^{(k)}$ es un individuo válido según la definición anterior. El algoritmo iterará sobre este **conjunto $P(t)$** aplicando operadores genéticos para generar la población de la siguiente generación, **$P(t+1)$** .

2.3 Función de aptitud (Fitness)

La función de aptitud determina la **calidad** de un individuo. Como el objetivo del problema de las N-Reinas es que ninguna reina se ataque con otra, buscaremos **minimizar el valor de esta función**. La solución óptima tendrá un fitness de **0**. Gracias a nuestra codificación basada en permutaciones, los ataques en filas y columnas están *descartados* por diseño. Por tanto, la función **$f(x)$** se encarga exclusivamente de contabilizar el **número de conflictos en las diagonales** del tablero. **Para un tablero $N \times N$, existen $2N-1$ diagonales ascendentes y $2N-1$ diagonales descendentes**. Las identificamos mediante un índice calculado a partir de la fila x_i y la columna i :

- Diagonal ascendente: índice $i + x_i$.
- Diagonal descendente: índice $(N - 1) - i + x_i$.

Sea $D_1[k]$ el número total de reinas presentes en la diagonal ascendente k , y $D_2[k]$ el número de reinas en la diagonal descendente k . La función penaliza cada diagonal que contenga más de una reina, sumando las **colisiones** (si hay q reinas en una diagonal, hay $q - 1$ colisiones que deben resolverse). Matemáticamente, el fitness **$f(x)$** de un individuo x se calcula así:

$$f(x) = \sum_{k=0}^{2N-2} \max(0, D_1[k] - 1) + \sum_{k=0}^{2N-2} \max(0, D_2[k] - 1)$$

Donde:

- $D_1[k] = \sum_{i=0}^{N-1} I(i + x_i = k)$
- $D_2[k] = \sum_{i=0}^{N-1} I((N - 1) - i + x_i = k)$

Siendo I una función que vale 1 si la condición es verdadera y 0 en caso contrario.

3 Metodologías y Estrategias Utilizadas

Como indica el título del informe, se ha utilizado un Algoritmo Genético para resolver el problema:

3.1 Algoritmo Genético

Los algoritmos genéticos son técnicas de búsqueda y optimización basadas en el Principio de Selección Natural de Darwin. Suelen ser usados como segunda opción o cuando el problema es excesivamente complicado, por ejemplo, cuando no es lineal y el espacio de búsqueda es muy extenso; este es el caso del problema de las N-Reinas: cuenta con un espacio de búsqueda astronómicamente grande, dada la cantidad de combinaciones en las casillas del tablero.

Este algoritmo siempre cuenta con el mismo patrón de pasos:

$$\text{selección} \rightarrow \text{cruce} \rightarrow \text{mutación}$$

En resumen:

1. Se genera una **población aleatoria** de n cromosomas.
2. Se evalúa el **fitness** de cada cromosoma según su función de aptitud $f(x)$.

3. Se crea una **nueva población** repitiendo los siguientes pasos hasta completarla:
 - (a) Se seleccionan dos cromosomas, ahora **progenitores**, de la población según un método de selección.
 - (b) Se cruzan los progenitores para formar nueva descendencia, dado el método de cruce elegido.
 - (c) Se **muta** la nueva descendencia con una probabilidad.
 - (d) Se **coloca** la nueva descendencia en la nueva población.
4. Se usa la nueva población generada para una **nueva ejecución** del algoritmo.
5. Si se cumple la **condición final**, detener y devolver la mejor solución en la población actual.
6. Volver al paso 2.

4 Implementación

El código ha sido programado en el lenguaje *Python*, usando la librería *DEAP*, vital para la implementación de algoritmos evolutivos como este.

```
# Toolbox se usa como contenedor para la definicion de los indices, individuos y
  poblacion

NB_QUEENS = 8
toolbox = base.Toolbox() # Se registra una funcion de generacion, que se llamara a
  la hora de crear individuos para que se generen numeros unicos, en un rango
  establecido

toolbox.register("indices", random.sample, range(NB_QUEENS), NB_QUEENS)

toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.
  indices)

toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

Listing 1.1. "Parámetros iniciales del algoritmo"

En esta parte del código se definen los **parámetros iniciales** necesarios para el correcto funcionamiento del algoritmo, usando *Toolbox* (de la librería DEAP) como contenedor.

La función de evaluación se define de la siguiente manera:

```
def FuncionEvaluacion(individual):
    tamanyo = len(individual)
    #Se tendran en cuenta dos diagonales, la que sube y la que baja
    diag_1 = [0] * (2 * tamanyo) #Esta es la que sube
    diag_2 = [0] * (2 * tamanyo) #Esta es la que baja

    for i in range(tamanyo):
        # individual[i] es la fila, i es la columna
        diag_1[i + individual[i]] += 1 #Aumenta el valor en esa posicion de la
            diagonal
        diag_2[tamanyo - 1 - i + individual[i]] += 1

    suma = 0
    #Ahora, se determinan los conflictos
    for i in range(2 * tamanyo):
        if diag_1[i] > 1:
            suma += diag_1[i] - 1
        if diag_2[i] > 1:
            suma += diag_2[i] - 1
```

```

# Tiene que devolver una tupla, porque la libreria solo acepta que la funcion
# devuelva una tupla
return suma,

#Una vez se tiene la funcion de evaluacion, se registra junto con las operaciones
toolbox.register("evaluate", FuncionEvaluacion)
toolbox.register("mate", tools.cxOrdered) #Operador de cruce
toolbox.register("mutate", tools.mutShuffleIndexes, indpb=2.0/NB_QUEENS) #Operador
# de mutacion
#El operador de mutacion lo que hace es intercambiar dos posiciones al azar de la
# lista
toolbox.register("select", tools.selTournament, tournsize=3) #El operador de
# seleccion usa la seleccion por torneo

```

Listing 1.2. "Función de evaluación"

Resumidamente, la función se enfoca en buscar conflictos en casillas mirando solo las diagonales; dado que cuando se crearon los individuos, se hicieron de manera que cada índice es una columna; mientras que cada número representa a una fila, es decir, en la posición 1 de la lista, si está el número 2 se refiere a la columna 1, fila 2. Luego se genera la población.

La ejecución del algoritmo se realiza mediante un bucle *While* que termina cuando se encuentra el **mínimo** (número mínimo de reinas que se atacan). El algoritmo sigue los mismos pasos explicados en la sección *Metodologías y Estrategias Utilizadas*.

5 Resultados Obtenidos

A continuación se explican en detalle los resultados obtenidos para tres experimentos:

5.1 Experimento para tablero 8x8 con 8 reinas

Para el primer experimento, optamos por el típico ejemplo pedagógico de 8 reinas con un tablero de 8x8 casillas. Se muestran a continuación los resultados en las figuras 1 y 2:

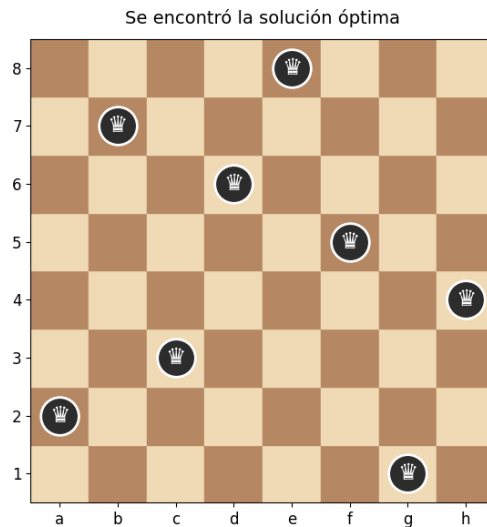


Fig. 1. Tablero solución para 8 reinas con tamaño 8x8

Encuentra correctamente una solución factible.

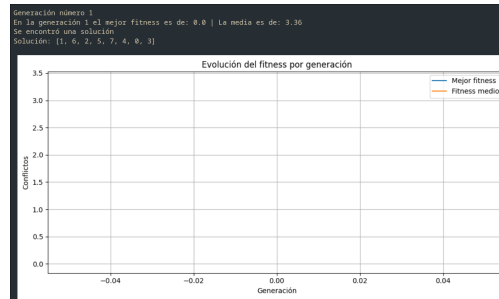


Fig. 2. Solución y gráfico de evolución del fitness

En este caso el algoritmo encuentra la solución factible en la **generación número 1**, por eso el gráfico con la evolución del fitness está **vacía**. El *tiempo de ejecución* es casi **inmediato**, y el algoritmo es extremadamente eficiente dado un problema tan **sencillo**.

5.2 Experimento para tablero de 20x20 casillas con 20 reinas

En este ejemplo se probó a **modificar el número de reinas**, pasando de **8**, que era el que se había establecido en un primer momento, a 20, con un tablero de 20x20. A priori no se modifica ningún otro parámetro, sólo *NB_QUEENS*. Se obtienen los siguientes resultados, reflejados en las figuras 3 y 4:

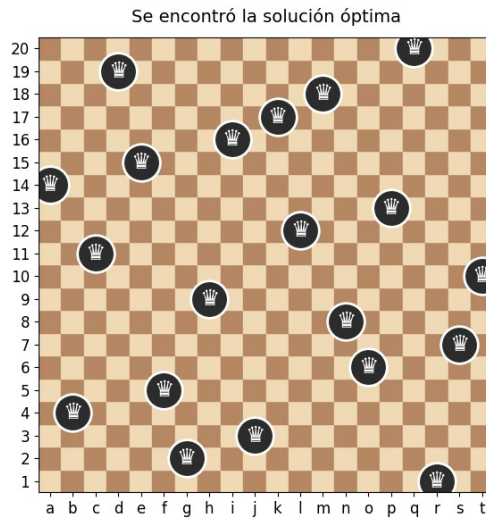


Fig. 3. Tablero solución para 20 reinas con tamaño 20x20

Como se puede ver, el algoritmo acaba encontrando una solución factible, donde no hay ningún par de reinas que se ataquen entre sí.

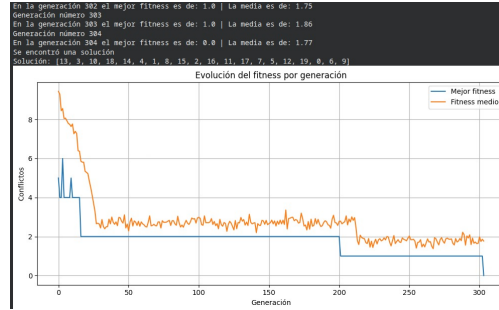


Fig. 4. Solución y gráfico de evolución del fitness

Los resultados revelan que se requiere un **número elevado de generaciones** para converger, así como un **tiempo de ejecución** más pronunciado; lo cual es lógico dada la exorbitante cantidad de **combinaciones** dentro del tablero.

Asimismo, la gráfica de la evolución del fitness muestra un gran **descenso del número de conflictos** en las primeras 30 generaciones aproximadamente, a partir de entonces disminuye relajadamente hasta llegar a 0. En resumen, a pesar de que el número de generaciones parece elevado, es realmente una cifra módica, demostrando el eficiente rendimiento del algoritmo.

5.3 Experimento para varias probabilidades de cruce y mutación con 16 reinas

Las siguientes figuras representan la evolución del fitness para 6 ejemplos con distintos valores de probabilidad en cruce y mutación, respectivamente.

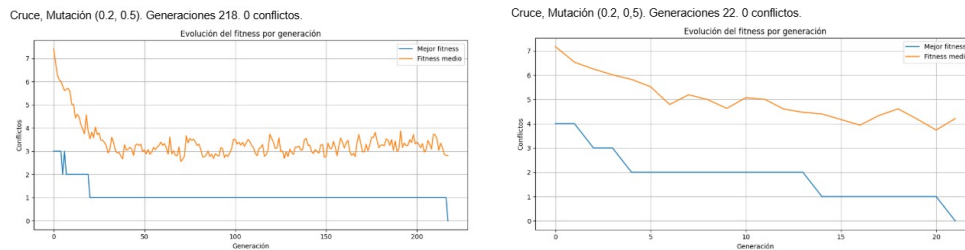


Fig. 5. Probabilidades cruce 0.2, mutación 0.5

Es importante puntualizar la Figura 5, porque sorprende ver distintos resultados para las **mismas probabilidades**. La diferencia entre estas se debe a la variación del algoritmo, ya que, al tener una **configuración inicial aleatoria**, la eficiencia también depende de la población inicial generada.

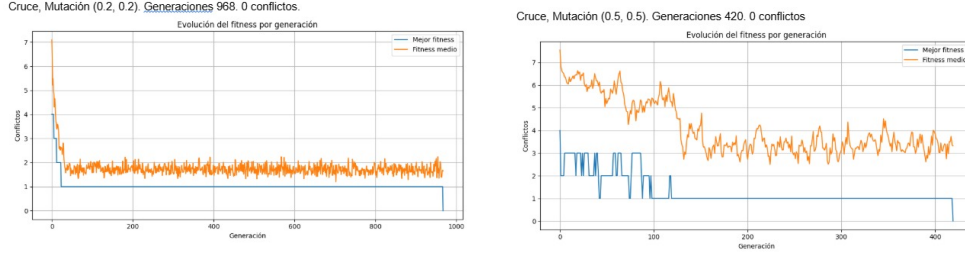


Fig. 6. Probabilidades (0.2,0.2) y (0.5,0.5)

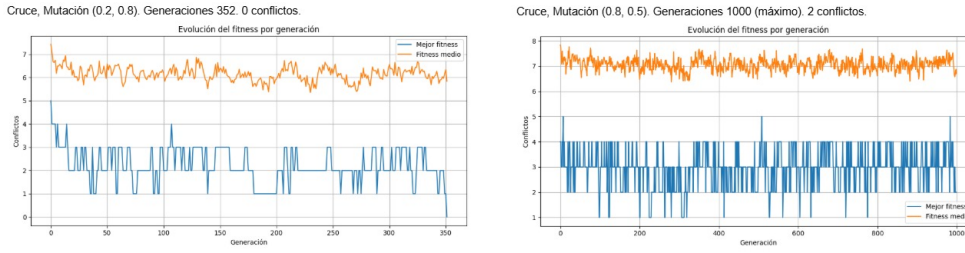


Fig. 7. Probabilidades (0.2,0.8) y (0.8,0.5)

Analizando con respecto a los resultados con $N=16$, podemos deducir que el **parámetro crítico es la probabilidad de cruce**. Como podemos observar, la eficiencia del algoritmo *varía* notablemente en función de esta probabilidad, incluso alcanzando las **1000 generaciones** (valor máximo establecido) con dos conflictos entre reinas. Cuanto más alta es la probabilidad de cruce más tiende el algoritmo a **destruir potenciales resultados** antes de que estos se establezcan.

Por otro lado, vemos que la probabilidad de mutación no es tan decisiva en comparación a la probabilidad de cruce. Comparando los distintos valores de este parámetro (0.2, 0.5, 0.8) observamos que un valor bajo 0.2 conduce a una **convergencia muy lenta**, como podemos observar, **968 generaciones**. En cambio, un valor alto (0.8) introduce **ruido** en el proceso, que dificulta la búsqueda del óptimo. El valor medio (0.5), que es el más balanceado, devuelve resultados de manera más eficientes (22 generaciones y 218 generaciones).

En resumen, para el caso de $N=16$, la configuración de cruce bajo 0.2 y mutación moderada 0.5 tiende a obtener mejores resultados, acercándose al óptimo real.

6 Discusión

Con respecto a los **dos primeros experimentos**, hallamos una **diferencia abismal** en el coste computacional: para $N=8$ el algoritmo *converge* en la generación 1, prácticamente de forma instantánea, mientras que para $N=20$ requiere del orden de cientos de generaciones y un tiempo de ejecución *notablemente mayor*. Esto es coherente con la **naturaleza combinatoria** del problema: el espacio de búsqueda crece de forma factorial con N , por lo que duplicar el número de reinas no duplica la dificultad, sino que **la multiplica exponencialmente**.

En ambos casos, no obstante, el algoritmo genético demuestra ser capaz de encontrar una solución factible (0 conflictos), lo que pone de manifiesto su **utilidad** frente a una búsqueda exhaustiva, que sería computacionalmente inviable para valores de N elevados.

El tercer experimento, con $N=16$, ofrece conclusiones más matizadas sobre el **impacto de los parámetros**. El parámetro más influyente es la probabilidad de cruce: valores altos (0.8) tienden a destruir buenas soluciones parciales **antes de que se consoliden**, impidiendo la convergencia incluso al llegar al límite de 1000 generaciones. Por el contrario, valores bajos (0.2) preservan mejor el material genético útil acumulado en cada generación.

La probabilidad de mutación actúa como un **mecanismo secundario** de exploración. Un valor *demasiado bajo* (0.2) **ralentiza** la salida de mínimos locales, mientras que un valor demasiado alto (0.8) introduce **ruido excesivo** que desestabiliza soluciones prometedoras. **El valor intermedio (0.5) logra el mejor equilibrio** entre exploración y explotación. La configuración óptima encontrada, cruce=0.2 y mutación=0.5, confirma que en este problema **conviene priorizar la explotación** sobre la exploración.

Finalmente, cabe destacar que, para las mismas probabilidades, el número de generaciones necesarias *varía* entre ejecuciones debido a la aleatoriedad de la población inicial. Esto evidencia que el rendimiento del algoritmo genético **no es determinista** y que, en aplicaciones reales, convendría promediar los resultados sobre varias ejecuciones independientes.

7 Conclusiones

En esta práctica se ha implementado un **algoritmo genético** haciendo uso de la librería **DEAP** para resolver el problema de las N-Reinas, obteniendo las siguientes conclusiones:

1. El algoritmo genético es capaz de **resolver el problema de las N Reinas de forma eficiente** para valores de N moderados (8 y 20), encontrando en ambos casos una solución con 0 conflictos.
2. **El coste computacional crece de forma pronunciada con N:** lo que para $N=8$ se resuelve en una sola generación, para $N=20$ requiere cientos de ellas. Aun así, el rendimiento es **notablemente superior** al de una búsqueda exhaustiva.
3. **La probabilidad de cruce es el hiperparámetro más crítico.** Valores elevados destruyen soluciones prometedoras antes de que maduren; valores bajos (0.2) permiten una convergencia más estable y rápida.
4. **La probabilidad de mutación tiene un efecto secundario pero relevante: actúa como mecanismo de diversidad genética.** El valor intermedio (0.5) resulta el más equilibrado entre exploración y explotación del espacio de soluciones.
5. **La estocasticidad inherente al algoritmo implica que los resultados varían entre ejecuciones con los mismos parámetros**, por lo que la elección de hiperparámetros debe evaluarse sobre múltiples ejecuciones para obtener conclusiones robustas.

8 Uso de IA

Durante el desarrollo de la práctica se ha utilizado **Claude** para los siguientes propósitos:

- Desarrollo de la función que muestra el gráfico con el tablero resultante *mostrar_tablero()*.
- Adquirir conocimiento y entendimiento sobre la librería **DEAP**.
- Redacción de la estructura y orden para las secciones *Discusión* y *Conclusiones*.



UNIVERSIDAD
DE MÁLAGA